

Access Persistence:

Persistence, bir təhlükəsizlik hadisəsində hücumçunun (və ya pentesterin) sistemə ilkin girişi əldə etdikdən sonra, bu girişi **sistem yenidən başladılsa, istifadəçi çıxış etsə, parol dəyişdirilsə belə** saxlaya bilmək qabiliyyətidir.

Niyə vacibdir?

İlkin giriş (initial access) çox vaxt qısa müddətli olur — məsələn, bir phishing linkinə klik və ya zəif bir servisdəki müvəqqəti sessiya. Hücumçu bu pəncərəni itirmək istəmir, ona görə də sistemdə "geri dönmə yolu" (backdoor/mexanizm) qurur.

Task0—0. Persistence Using Startup Folder :

Startup —>

Startup folder (Windows-da **Başlanğıc** qovluğu) persistence texnikalarının ən sadə və klassik nümunələrindən biridir.

- Press **Win + R**, type **shell:startup** → opens the current user's folder
- Press **Win + R**, type **shell:common startup** → opens the global (all users) folder

İki əsas yol var:

1. İstifadəçiyə xas (per-user):

```
C:\Users\<istifadəçi>\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup
```

Yalnız həmin istifadəçi login olduqda işə düşür.

1. Bütün istifadəçilər üçün (all-users, admin tələb edir):

```
C:\ProgramData\Microsoft\Windows\Start Menu\Programs\Startup
```

Kompüterə giriş edən **hər hansı istifadəçi** üçün işə düşür — daha güclü persistence, çünki admin başqa hesabla da login olsa belə tetiklənir.

Necə istifadə olunur (konseptual)

Hücumçu bu qovluğa məsələn bir `.lnk` (shortcut) faylı və ya birbaşa `.exe` / `.bat` / `.vbs` script yerləşdirir. Fayl bir C2 (command-and-control) əlaqəsi quran zərərli koda işarə edə bilər — beləliklə hər restart-da hücumçu yenidən "əl sıxışma" (beacon) alır.

User-specific Startup folder (`...\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup`)

On a clean, freshly-installed system, this is usually **empty**. It only gets populated when:

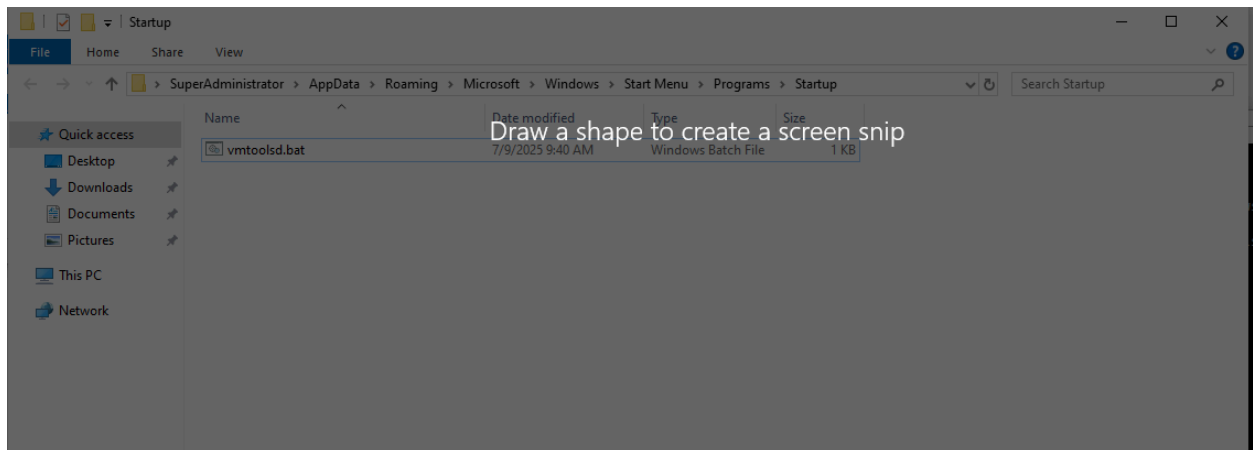
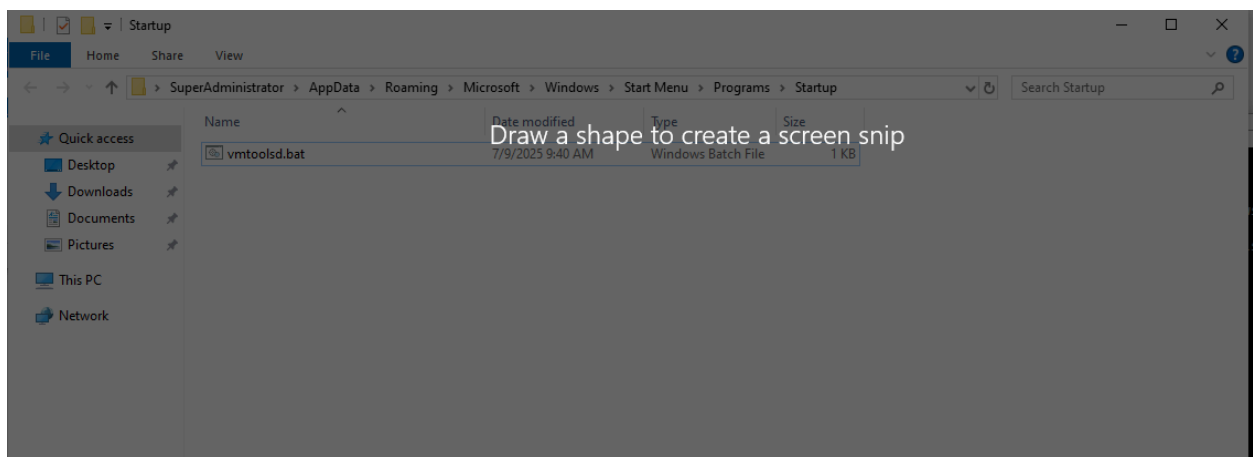
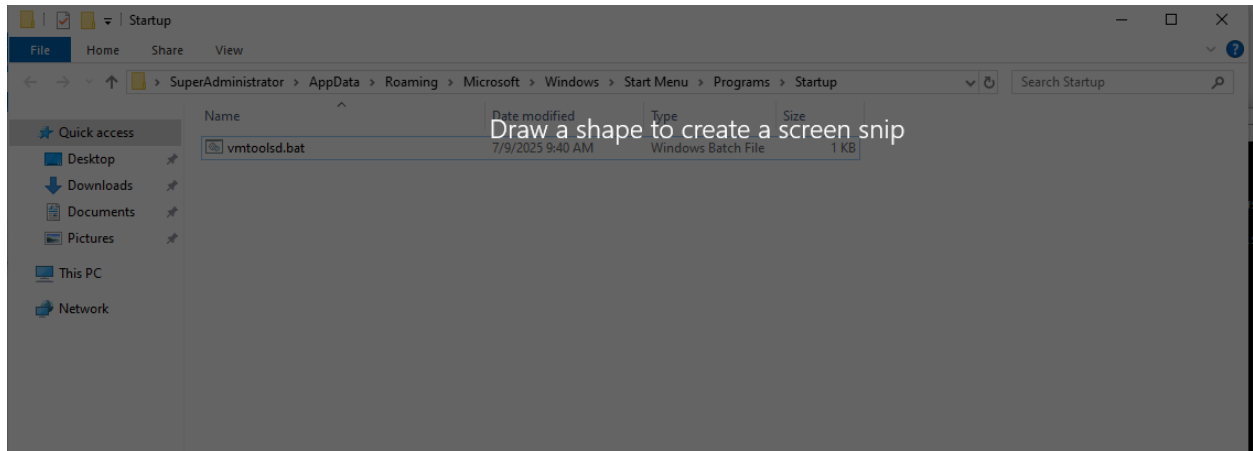
- The user (or an installer) deliberately adds a shortcut to an app they want auto-launched at login (e.g. Dropbox, Discord, a note-taking app, OneDrive)
- Some legitimate software installers drop a `.lnk` here during setup

So if this is a lab VM that hasn't had extra software installed, you'd expect to see **nothing at all** — which makes any file here suspicious by default.

Global Startup folder (`C:\ProgramData\Microsoft\Windows\Start Menu\Programs\Startup`)

Also typically **empty** on a default Windows install. It's used by:

- Enterprise software (antivirus agents, VPN clients, monitoring tools) that need to run for every user
- Some legitimate system utilities installed with admin rights



Task1—1. Persistence Using Registry

Autorun :

. Open Registry Editor

Press `Win + R` , type `regedit` , hit Enter (elevate as SuperAdministrator if prompted — `Root@123`)

Taskın məqsədi

Bu tapşırıq **Windows Registry vasitəsilə persistence (davamlılıq) texnikasını** öyrənmək üçün idi. Real hücumçular sistemə bir dəfə giriş əldə etdikdən sonra, həmin girişi itirmək istəmirlər — kompüter yenidən işə salınsa belə, öz kodlarının avtomatik işə düşməsinə təmin etmək istəyirlər. Bunun üçün Windows Registry-də müəyyən "Run" açarlarından istifadə edirlər ki, hər dəfə istifadəçi login olanda müəyyən proqram/skript avtomatik başlasın.

Necə həll etdik — addım-addım

1. Registry-də şübhəli girişi tapdıq

`regedit` açıb bu path-ə getdik:

```
HKEY_CURRENT_USER\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
```

Bu, Windows-un "avtomatik başlat" siyahısıdır — burada nə varsa, istifadəçi login olanda avtomatik işə düşür. Orada `flag2` adlı normal olmayan bir giriş gördük ki, PowerShell-i çağıraraq bir `.ps1` skriptini işə salırdı.

2. Skriptin ünvanını tapıb açdıq

Registry girişinin "Data" hissəsində skriptin tam yolu var idi:

```
C:\Program Files (x86)\WindowsPowerShell\Modules\PackageManagement\1.0.0.1\UserProfile\UserProfile.ps1
```

Bu faylı **işə salmadan**, sadəcə Notepad ilə açıb daxilini oxuduq (əgər double-click edib açsaydıq, skript işə düşərdi).

3. Skriptin daxilini analiz etdik

Skriptdə üç hissə var idi:

- Yalan/saxta flag yazan sətir (`Set-Content ... "Holberton{XXX}"`) — bu bizi çaşdırmaq üçün qoyulmuşdu
- `calc.exe` işə salan sətir — sadəcə persistence-in real işlədiyini göstərmək üçün (real hücumda bunun yerinə zərərli proqram olardı)
- Əsl flag isə **hex kodlu byte array** şəklində gizlədilmişdi:

```
$a=0x48,0x6f,0x6c,0x62,...[System.Text.Encoding]::ASCII.GetString($a) | Write-Output
```

4. Hex-i decode etdik

Hər hex dəyəri bir ASCII simvoluna uyğun gəlir (məs: `0x48` = "H", `0x6f` = "o" və s.). Bunları sıra ilə çevirəndə çıxan söz:

```
Holberton{Persist_Through_Win_Register}
```

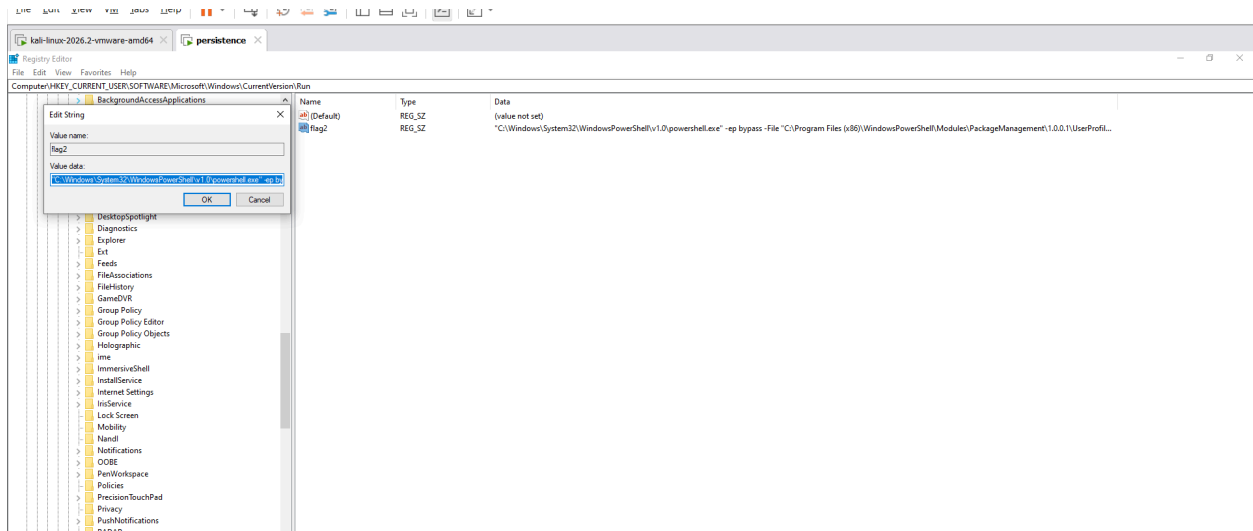
Nəyə görə vacibdir (cleanup hissəsi)

Task həm də vurğulayır ki, pentester/analitik olaraq iş bitəndən sonra **bütün izləri təmizləməliyik**:

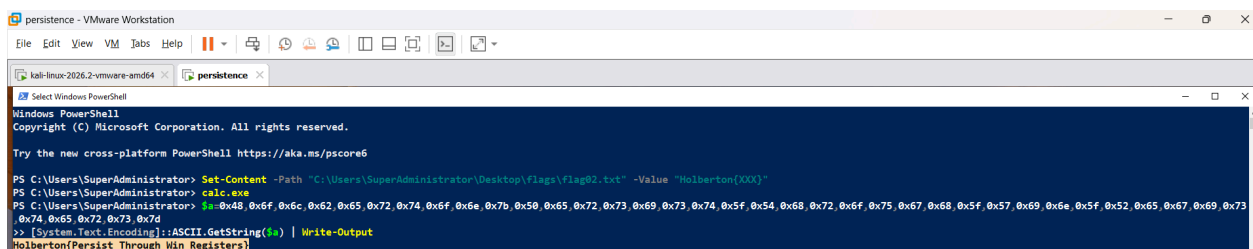
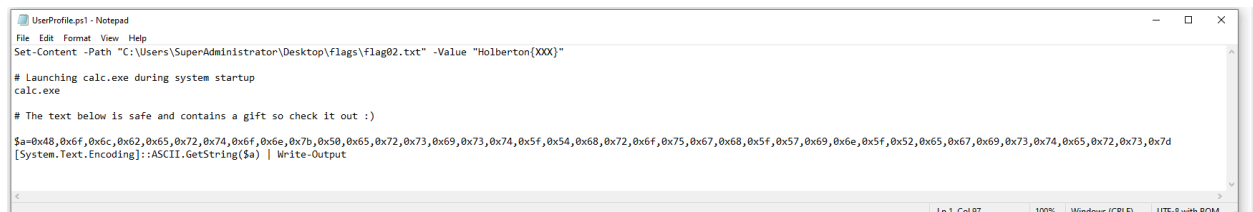
- Registry-dəki `flag2` girişini silmək
- `.ps1` faylını disk-dən silmək
- PowerShell tarixçəsini (`ConsoleHost_history.txt`) təmizləmək

Bu, real dünyada "blue team" (müdafiə tərəfi) təcrübəsi üçün vacibdir — sistemin əvvəlki vəziyyətinə qaytarılması və hər hansı iz buraxılmaması bacarığını inkişaf etdirir.

Checker qəbul etmirsə, əvvəlki mesajımda göstərdiyim kimi, ehtimal ki, `1-flag.txt` faylında görünməyən boşluq/yeni sətir və ya BOM problemi var — string-in özü düzgündür.



File Edit Format View Help
 "C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -ep bypass File "C:\Program Files (x86)\WindowsPowerShell\Modules\PackageManagement\1.0.0.1\UserProfile.ps1"



Task2—2. Persistence Using Services :

1. Open Services manager

Press **Win + R** , type **services.msc** , hit Enter.

Taskın məqsədi

Bu tapşırıq **Windows Services (xidmətlər) vasitəsilə persistence** texnikasını öyrənmək üçündür. Windows-da services arxa planda, istifadəçi login olmasa belə, sistem başlayanda avtomatik işə düşür. Bu, hücumçular üçün çox əlverişli bir mexanizmdir çünki:

- İstifadəçi görmür (arxa planda işləyir)
- Sistem restart olsa belə davam edir
- Adi proseslərlə qarışb gizlənə bilir

Ona görə real hücumçular çox vaxt zərərli kodlarını "service" kimi qeydiyyatdan keçirirlər ki, uzun müddət sistemdə gizli qala bilsinlər.

Necə həll edəcəyik — addım-addım

1. Services idarəetməsini açırıq Win + R → `services.msc` yazıb Enter.

2. `flag3` adlı service-i tapırıq

Siyahıda "f" hərfinə keçib `flag3` adlı qeyri-adi service-i axtarıq — normal Windows service-ləri arasında bu, açıq-aydın fərqlənəcək.

3. Service-in Description sahəsinə baxırıq

Bu addım vacibdir çünki tapşırığa görə **base64 ilə kodlanmış parol** məhz Description sahəsində gizlədilib. GUI-də bu sahə bəzən kəsilə bilər (truncate olur), ona görə tam mətni görmək üçün PowerShell-dən istifadə etmək daha etibarlıdır:

```
Get-CimInstance -ClassName Win32_Service -Filter "Name='flag3'" | Select-Object Name, DisplayName, Description, PathName
```

4. Base64-ü decode edirik

Tapılan base64 string-i buradan çevirmək üçün:

```
[System.Text.Encoding]::UTF8.GetString([System.Convert]::FromBase64String("BURAYA_BASE64"))
```

Çıxan nəticə flag olacaq (əvvəlki formatlara bənzər, `Holberton{...}` şəklində ola bilər).

Nəyə görə vacibdir

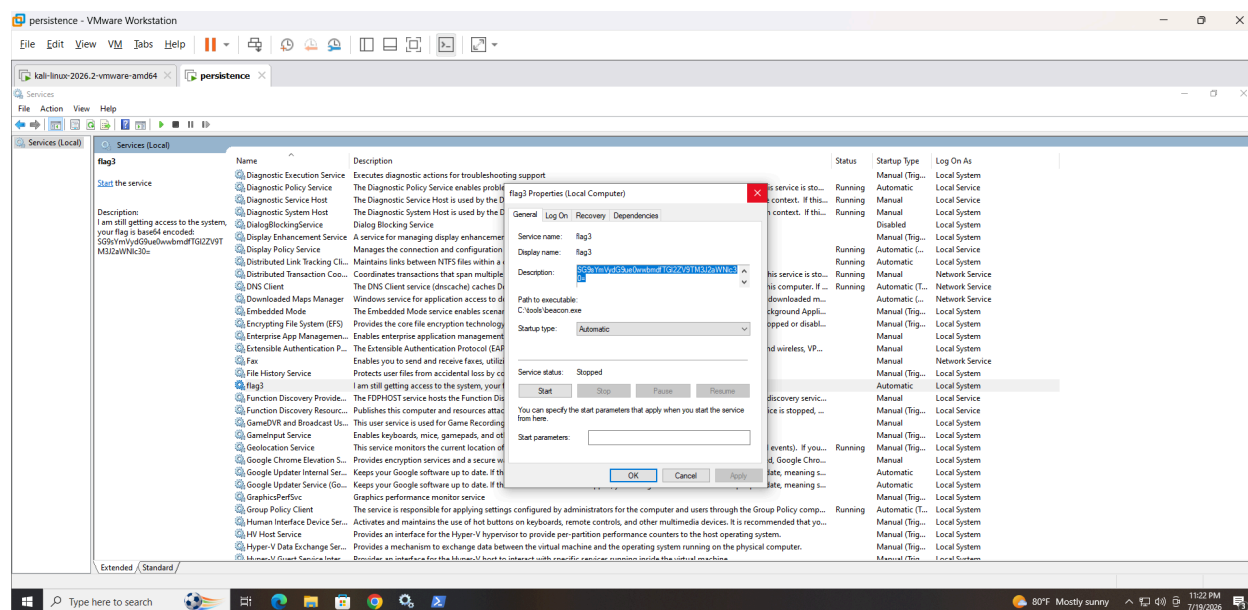
Bu üsul real hücum ssenarilərində çox işlədilir — məsələn, ransomware və ya RAT (Remote Access Trojan) qrupları məhz belə "gizli service"lər vasitəsilə uzun müddət aşkarlanmadan sistemdə qalırlar. Bu taskı öyrənməklə siz həm hücum məntiqini, həm də **blue team** kimi belə şübhəli service-ləri necə aşkar edəcəyinizi (`Get-CimInstance`, `services.msc` audit) başa düşürsünüz.

Son olaraq, cleanup mərhələsi də vacibdir:

```
Stop-Service -Name "flag3"  
sc.exe delete flag3
```

Bu, sistemi əvvəlki, təmiz vəziyyətinə qaytarır — real audit/pentest işlərində iz buraxmamaq peşəkarlığın vacib hissəsidir.

Hazır olanda `Get-CimInstance` əmrini işə salıb nəticəni mənə göndər, birlikdə decode edərək.



Decode from Base64 format

Simply enter your data then push the decode button.

SG9sYmVydG9ue0wwbmdFTGI2ZV9TM3J2aWNlc30=

For encoded binaries (like images, documents, etc.) use the file upload form a little further down on this page.

UTF-8 Source character set.

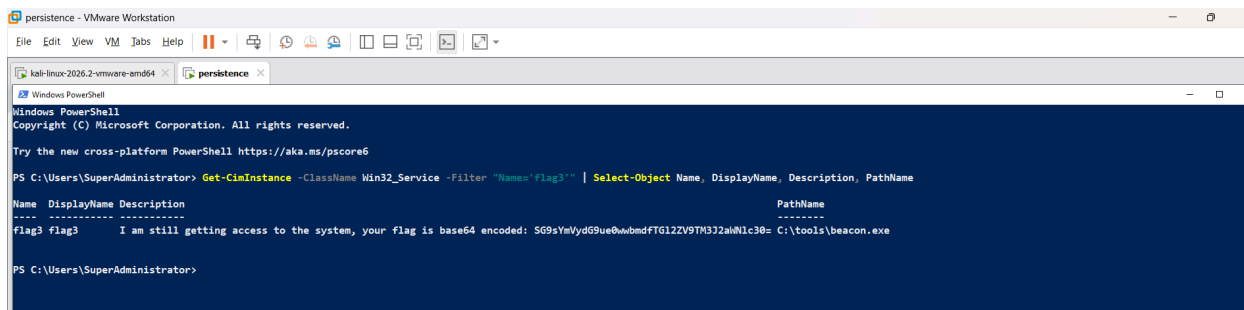
☐ Decode each line separately (useful for when you have multiple entries).

☒ Live mode OFF Decodes in real-time as you type or paste (supports only the UTF-8 character set).

< DECODE > Decodes your data into the area below.

Holberton(L0ng_Live_S3rvices)

OR



```
persistence - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2026.2-vmware-amd64 persistence
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Users\SuperAdministrator> Get-CimInstance -ClassName Win32_Service -Filter "Name='flag3'" | Select-Object Name, DisplayName, Description, PathName

Name      DisplayName Description                                     PathName
-----
flag3     flag3      I am still getting access to the system, your flag is base64 encoded: SG9sYmVydG9ue0wwbmdFTGI2ZV9TM3J2aWNlc30= C:\tools\beacon.exe

PS C:\Users\SuperAdministrator>
```

Task3—3. Persistence Using Scheduled Tasks :

1. Open Task Scheduler

- Win + R → type `taskschd.msc` → Enter

- Or search "Task Scheduler" in the Start menu

Taskın məqsədi

Bu tapşırıq **Windows Scheduled Tasks (Tapşırıq Planlayıcısı)** vasitəsilə persistence texnikasını öyrənmək üçündür. Windows-da Task Scheduler, müəyyən vaxtlarda, ya da müəyyən hadisələr (məsələn, sistem başlayanda və ya istifadəçi login olanda) baş verəndə avtomatik proqram/skript işə salmaq üçün nəzərdə tutulub — normalda bu, sistem update-ləri, backup skriptləri kimi qanuni işlər üçün istifadə olunur.

Amma hücumçular da eyni mexanizmi öz məqsədləri üçün istismar edə bilirlər:

- Zərərli skriptini "trigger: at logon" və ya "at startup" ilə konfigurasiya edirlər
- Beləliklə, sistem hər dəfə açılanda və ya istifadəçi login olanda, onların kodu sakitcə arxa planda işə düşür
- Task Scheduler-də yüzlərlə legitim (qanuni) task olduğu üçün, zərərli task da bunların arasında gizlənə bilər — bu da onu aşkar etməyi çətinləşdirir

Bu taskın "Target Task Configuration" hissəsində qeyd olunan **Triggers: System startup or user logon** da məhz bunu göstərir — real hücumçuların ən çox istifadə etdiyi trigger tipləridir, çünki bunlar sistemi hər restart/login-də zərərli kodun işə düşməsinə təmin edir.

Necə həll edəcəyik — addım-addım

1. Task Scheduler-i açırıq **Win + R** → **taskschd.msc** → Enter

2. Task-ı axtarıq

GUI-də əl ilə axtarmaq əvəzinə, PowerShell ilə bütün sistemdəki task-ları bir dəfəyə axtarmaq daha effektivdir:

```
Get-ScheduledTask | Where-Object { $_.TaskName -like "*flag*" }
```

Bu, adında "flag" olan task-ı harada olursa-olsun (istənilən qovluqda) tapacaq.

3. Task-ın Description sahəsini oxuyuruq

Tapdıqdan sonra, əvvəlki task-lardan fərqli olaraq, bu dəfə flag **birbaşa, açıq**

şəkildə (plaintext) Description sahəsində yazılıb — decode etməyə ehtiyac yoxdur:

```
(Get-ScheduledTask -TaskName "TAPILAN_AD").Description
```

4. Triggers və Actions tab-larına da baxırıq

GUI-də task-ın üstünə klik edib, aşağı paneldə **Triggers** (login/startup-da işə düşdüyünü təsdiqləyir) və **Actions** (hansı proqram/skripti işə saldığını göstərir) tab-larını yoxlayırıq — bu, real hücum ssenarisini anlamaq üçün vacibdir.

Nəyə görə vacibdir

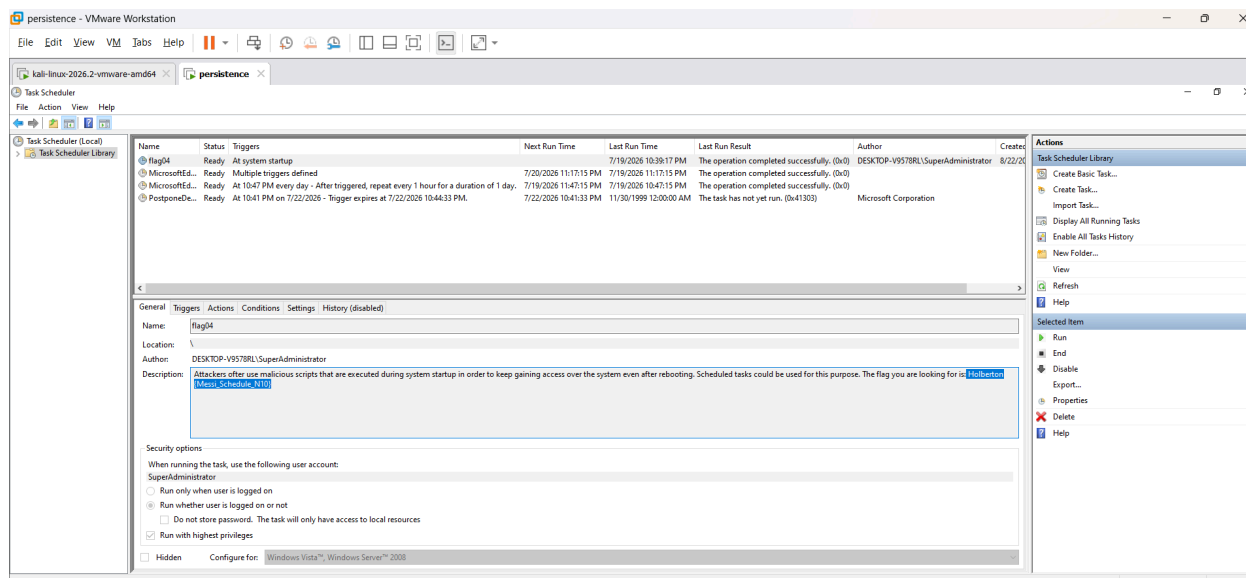
Real dünyada, blue team (müdafiə) analitikləri məhz bu üsulla — yəni bütün scheduled task-ları müntəzəm audit edərək — sistemə gizlicə yerləşdirilmiş persistence mexanizmlərini aşkar edirlər. Xüsusilə "at logon"/"at startup" trigger-li, tanınmayan və ya şübhəli adlı task-lar həmişə diqqətlə yoxlanmalıdır.

Cleanup (son addım)

```
Unregister-ScheduledTask -TaskName "TAPILAN_AD" -Confirm:$false
```

Bu, task-ı sistemdən tamamilə silir və mühiti təmiz vəziyyətə qaytarır.

İndi hazır olanda `Get-ScheduledTask | Where-Object { $_.TaskName -like "*flag*" }` əmrini icra et və nəticəni mənə göndər — birlikdə davam edərik.



Task4—4. Persistence Using BITSAdmin :

Persistence Using BITS (Background Intelligent Transfer Service)

The Background Intelligent Transfer Service (BITS) is a legitimate Windows component designed to transfer files in the background using idle network bandwidth. While it is commonly used for system updates and application downloads, attackers can abuse BITS to establish persistence and execute malicious payloads in a stealthy manner.

Because BITS jobs can survive system reboots, retry failed transfers, and operate silently in the background, they provide an effective mechanism for maintaining long-term access to a compromised system.

Understanding BITS and Its Capabilities

BITS is designed to:

- Transfer files asynchronously in the background
- Automatically retry failed downloads
- Resume interrupted transfers
- Operate with minimal user visibility

These characteristics make BITS particularly attractive to attackers who want to maintain persistence without raising suspicion.

Enumerate existing jobs first:

powershell

```
bitsadmin/list/allusers/verbose
```

```
PS C:\Users\SuperAdministrator> bitsadmin /list /allusers /verbose

BITSADMIN version 3.0
BITS administration utility.
(C) Copyright Microsoft Corp.

Unable to enum jobs - 0x80070005
Access is denied.
```

Create the job — downloading a small benign test file and setting completion to launch `calc.exe` as your safe stand-in payload:

cmd

```
bitsadmin /create demoJob #create the job
bitsadmin /addfile demoJob "https://raw.githubusercontent.com/microsoft/vscode/main/README.md" "C:\Windows\Temp\demo_payload.txt" #A file was added to the job for download
bitsadmin /SetNotifyCmdLine demoJob "C:\Windows\System32\calc.exe" NULL #Once the file was successfully attached, a notification command was configured to execute a program upon completion
```

```
bitsadmin /SetNotifyFlags demoJob 3
bitsadmin /resume demoJob #The job was started
```

```
PS C:\Users\SuperAdministrator> bitsadmin /create demoJob
>> bitsadmin /addfile demoJob "https://raw.githubusercontent.com/microsoft/vscode/main/README.md" "C:\Windows\Temp\demo_payload.txt"
>> bitsadmin /SetNotifyCmdLine demoJob "C:\Windows\System32\calc.exe" NULL
>> bitsadmin /SetNotifyFlags demoJob 3
>> bitsadmin /resume demoJob

BITSADMIN version 3.0
BITS administration utility.
(C) Copyright Microsoft Corp.

Created job {52C4E771-9764-4857-B4E7-559115A0E277}.

BITSADMIN version 3.0
BITS administration utility.
(C) Copyright Microsoft Corp.

Found 3 jobs named "demoJob".
Use the job identifier instead of the job name.

BITSADMIN version 3.0
BITS administration utility.
(C) Copyright Microsoft Corp.

Found 3 jobs named "demoJob".
Use the job identifier instead of the job name.

BITSADMIN version 3.0
BITS administration utility.
(C) Copyright Microsoft Corp.

Found 3 jobs named "demoJob".
Use the job identifier instead of the job name.
```

```
PS C:\Users\SuperAdministrator> bitsadmin /resume demoJob

BITSADMIN version 3.0
BITS administration utility.
(C) Copyright Microsoft Corp.

Found 3 jobs named "demoJob".
Use the job identifier instead of the job name.
```

- `SetNotifyFlags 3` tells BITS to fire the notify command on job completion or error, which is what triggers the "execution" step.
- Swap the download URL for anything harmless you control (or a public static file); the point being demonstrated is the *mechanism*, not the payload itself.

Check status:

powershell

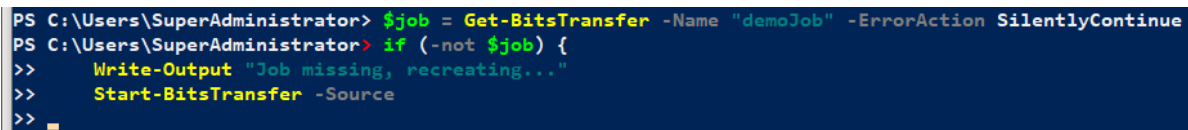
```
bitsadmin/info demoJob/verbose
```

Enhancing Persistence with Monitoring

To ensure persistence even if the BITS job is removed, attackers may deploy a monitoring script. A simple PowerShell checker can verify whether the job exists and recreate it if necessary:

```
$jobName = "demoJob"
$job = Get-BitsTransfer -Name $jobName -ErrorAction SilentlyContinue

if (-not $job) {
    Write-Output "Job missing, recreating..."
    Start-BitsTransfer -Source
```



```
PS C:\Users\SuperAdministrator> $job = Get-BitsTransfer -Name "demoJob" -ErrorAction SilentlyContinue
PS C:\Users\SuperAdministrator> if (-not $job) {
>>     Write-Output "Job missing, recreating..."
>>     Start-BitsTransfer -Source
>> }
```

Detection and Prevention

Defenders can identify suspicious BITS activity by inspecting active jobs:

```
bitsadmin /list /allusers /verbose
```

Additionally, BITS activity can be monitored through Windows Event Logs:

```
Event Viewer → Applications and Services Logs → Microsoft → Windows → Bits-Client
```

Indicators of compromise may include:

- Unknown or suspicious download sources
- Unexpected execution of binaries after downloads

- Unusual job names or descriptions